CAHIER DES CHARGES

Projet 7 — Plateforme d'Authentification Multi-Facteurs

Méthode SCRUM appliquée

IFRI — Université d'Abomey-Calavi
Année académique 2024-2025
Mars 2025

1. Présentation générale du projet

1.1 Contexte

Dans un contexte de sécurité informatique croissante, les systèmes d'authentification classiques basés sur un simple couple identifiant/mot de passe ne sont plus suffisants pour garantir la sécurité des accès aux plateformes numériques. Le vol de mots de passe, les attaques par force brute et le phishing constituent des menaces réelles et fréquentes.

C'est dans ce cadre que s'inscrit le Projet 7 : développer une plateforme web d'authentification multi-facteurs (MFA — Multi-Factor Authentication) qui renforce la sécurité des connexions en ajoutant une couche de vérification supplémentaire après la saisie du mot de passe, via l'envoi d'un code de vérification par e-mail et/ou WhatsApp.

1.2 Objectifs du projet

Les objectifs du projet sont les suivants :

- Permettre une authentification sécurisée par login et mot de passe.
- Générer et envoyer un code OTP (One-Time Password) à l'utilisateur via e-mail et/ou WhatsApp après la première étape d'authentification.
- Valider l'accès à la plateforme uniquement après saisie correcte du code OTP.
- Conserver un historique détaillé des connexions (adresse IP, date, heure).
- Offrir une interface utilisateur moderne, intuitive et responsive.
- Assurer la sécurité et la confidentialité des données des utilisateurs.

1.3 Utilisateurs cibles

La plateforme s'adresse à plusieurs catégories d'utilisateurs :

Type d'utilisateur	Rôle
Utilisateur final	Se connecte à la plateforme avec identifiant + mot de passe, puis valide le code OTP reçu.
Administrateur	Gère les comptes, consulte les historiques de connexion, configure les paramètres de sécurité.
Super Administrateur	Dispose de tous les droits d'administration et peut réinitialiser les données sensibles.

1.4 Périmètre du projet

Le projet couvre les fonctionnalités suivantes, conformément au cahier de charges fourni par le commanditaire :

- Module d'authentification (login + mot de passe).
- Module OTP (génération + envoi par e-mail et WhatsApp).
- Module de validation du code OTP.
- Module d'historique des connexions.
- Interface d'administration.

Le projet n'inclut pas : la gestion des droits métiers (rôles applicatifs), ni l'intégration avec des systèmes tiers au-delà de l'e-mail et de WhatsApp.

2. Méthodologie adoptée : SCRUM

2.1 Justification du choix de SCRUM

La méthode SCRUM a été choisie pour les raisons suivantes :

- Adaptabilité : SCRUM permet d'intégrer facilement des modifications ou évolutions des besoins en cours de développement, ce qui est particulièrement pertinent dans un projet académique où les exigences peuvent être précisées au fil des sprints.
- Livraisons incrémentales : chaque Sprint produit un livrable fonctionnel et testable, permettant des validations régulières avec le commanditaire (le professeur).
- Collaboration : SCRUM favorise la communication entre les membres de l'équipe et avec le Product Owner, réduisant les risques de mauvaise interprétation des besoins.
- Transparence : les outils SCRUM (backlog, sprint board, rétrospectives) offrent une visibilité claire sur l'avancement du projet.
- Convenance académique : SCRUM se prête bien aux projets de courte durée avec des équipes de taille limitée, typique du cadre universitaire.

2.2 Organisation de l'équipe SCRUM

Rôle SCRUM	Membres	Responsabilités
Product Owner	Le Professeur	Définit les besoins, valide les livrables à chaque Sprint, priorise le backlog.
Scrum Master	*[2 à 3 membres du groupe]*	Facilite les cérémonies SCRUM, lève les obstacles, garantit l'application de la méthode.
Scrum Team	*[Reste du groupe]*	Développement, tests, documentation. Auto-organisée et pluridisciplinaire.
Management	Scrum Master + Product Owner	Décisions stratégiques sur l'orientation du projet et validation finale.

2.3 Cérémonies SCRUM planifiées

- Sprint Planning : réunion de planification au début de chaque Sprint (définition des tâches et estimations).
- Daily Scrum : réunion quotidienne courte (15 min) pour synchroniser l'équipe et identifier les blocages.
- Sprint Review : présentation du livrable au Product Owner en fin de Sprint pour validation.
- Sprint Retrospective : réflexion collective sur ce qui a bien fonctionné et ce qui peut être amélioré.

3. Fonctionnalités du système

3.1 Description des fonctionnalités principales

N°	Fonctionnalité	Description détaillée
1	**Authentification par login/mot de passe**	L'utilisateur saisit son identifiant et son mot de passe. Le système vérifie les informations contre la base de données en utilisant un hashage SHA256. En cas d'échec répété, le compte est temporairement verrouillé.
2	**Génération du code OTP**	Après validation du mot de passe, le système génère automatiquement un code OTP à 6 chiffres avec une durée de validité de 10 minutes.
3	**Envoi du code par e-mail**	Le code OTP est envoyé à l'adresse e-mail enregistrée de l'utilisateur via un service SMTP (ex : PHPMailer).
4	**Envoi du code par WhatsApp**	Le code OTP est également envoyé via l'API WhatsApp Business Whapi (gate.whapi.cloud).
5	**Validation du code OTP**	L'utilisateur saisit le code reçu dans un formulaire dédié. Le système vérifie la validité du code (correspondance + délai d'expiration). Si le code est expiré, un renvoi est possible.
6	**Historique des connexions**	Chaque tentative de connexion (réussie ou échouée) est enregistrée avec : l'adresse IP, la date, l'heure, le statut (succès/échec), et le canal OTP utilisé.
7	**Tableau de bord administrateur**	L'administrateur peut consulter les historiques de connexion, gérer les comptes utilisateurs, et configurer les paramètres (durée OTP, nombre de tentatives, etc.).
8	**Déconnexion sécurisée**	Invalidation complète de la session utilisateur à la déconnexion.

3.2 Contraintes techniques

- Sécurité des mots de passe : hashage avec SHA256 (via la fonction hash() native PHP).
- Codes OTP : générés de façon cryptographiquement sûre (random_int en PHP).
- HTTPS obligatoire en production pour protéger les données en transit.
- Protection contre les injections SQL via l'utilisation de requêtes préparées (PDO).
- Conformité aux bonnes pratiques OWASP (Top 10 des vulnérabilités web).
- Interface responsive compatible mobile et desktop.

4. Analyse UML

4.1 Diagramme de cas d'utilisation (Use Case)

Le diagramme ci-dessous représente les acteurs du système et leurs interactions avec les cas d'utilisation de la plateforme MFA. Les acteurs internes sont l'Utilisateur et l'Administrateur. Les acteurs externes sont le Système e-mail (SMTP) et l'API WhatsApp. Les stéréotypes «include» et «extend» modélisent les dépendances entre cas d'utilisation.

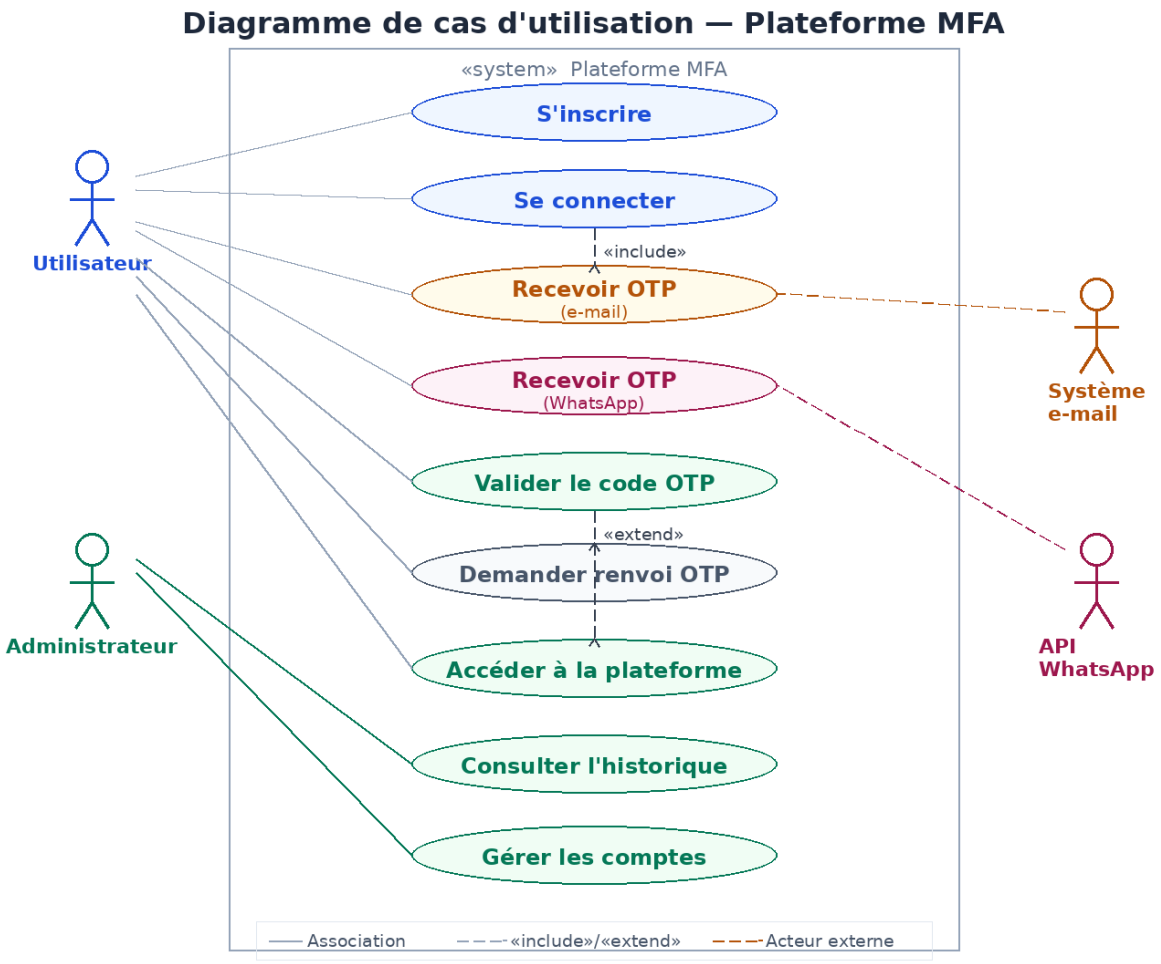


Figure 1 — Diagramme de cas d'utilisation de la plateforme MFA

4.2 Diagramme de classes

Le diagramme de classes ci-dessous présente les 5 classes du système, leurs attributs (visibilité, types, clés), leurs méthodes ainsi que les relations entre elles : héritage (Administrateur étend Utilisateur), associations 1..N (Utilisateur génère OTPCode et enregistre HistoriqueConnexion), et dépendances (OTPCode et Administrateur utilisent Parametres).

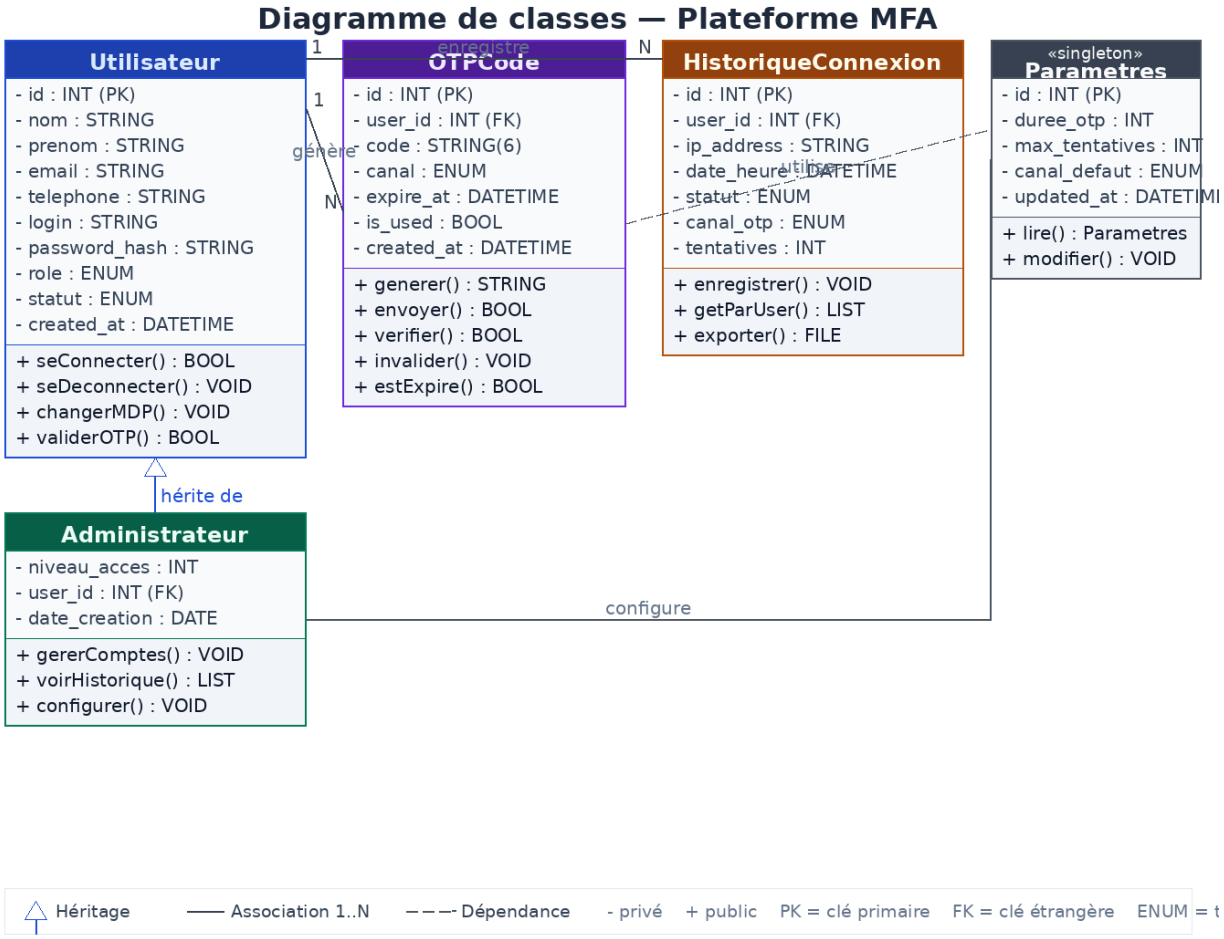


Figure 2 — Diagramme de classes de la plateforme MFA

Relations entre classes :

- Utilisateur (1) ──< (N) OTPCode : un utilisateur peut avoir plusieurs codes OTP générés.
- Utilisateur (1) ──< (N) HistoriqueConnexion : un utilisateur possède un historique de connexions.
- Administrateur hérite de Utilisateur (généralisation).
- Parametres est une classe singleton gérée par l'administrateur.

4.3 Diagramme de séquence MFA

Le diagramme de séquence ci-dessous modélise l'ensemble des échanges entre les acteurs du système lors d'une authentification MFA complète. Il couvre trois flux distincts : le flux principal (authentification réussie), les flux d'erreur (mot de passe incorrect, compte bloqué, code OTP invalide) et le flux d'expiration OTP avec renvoi du code.

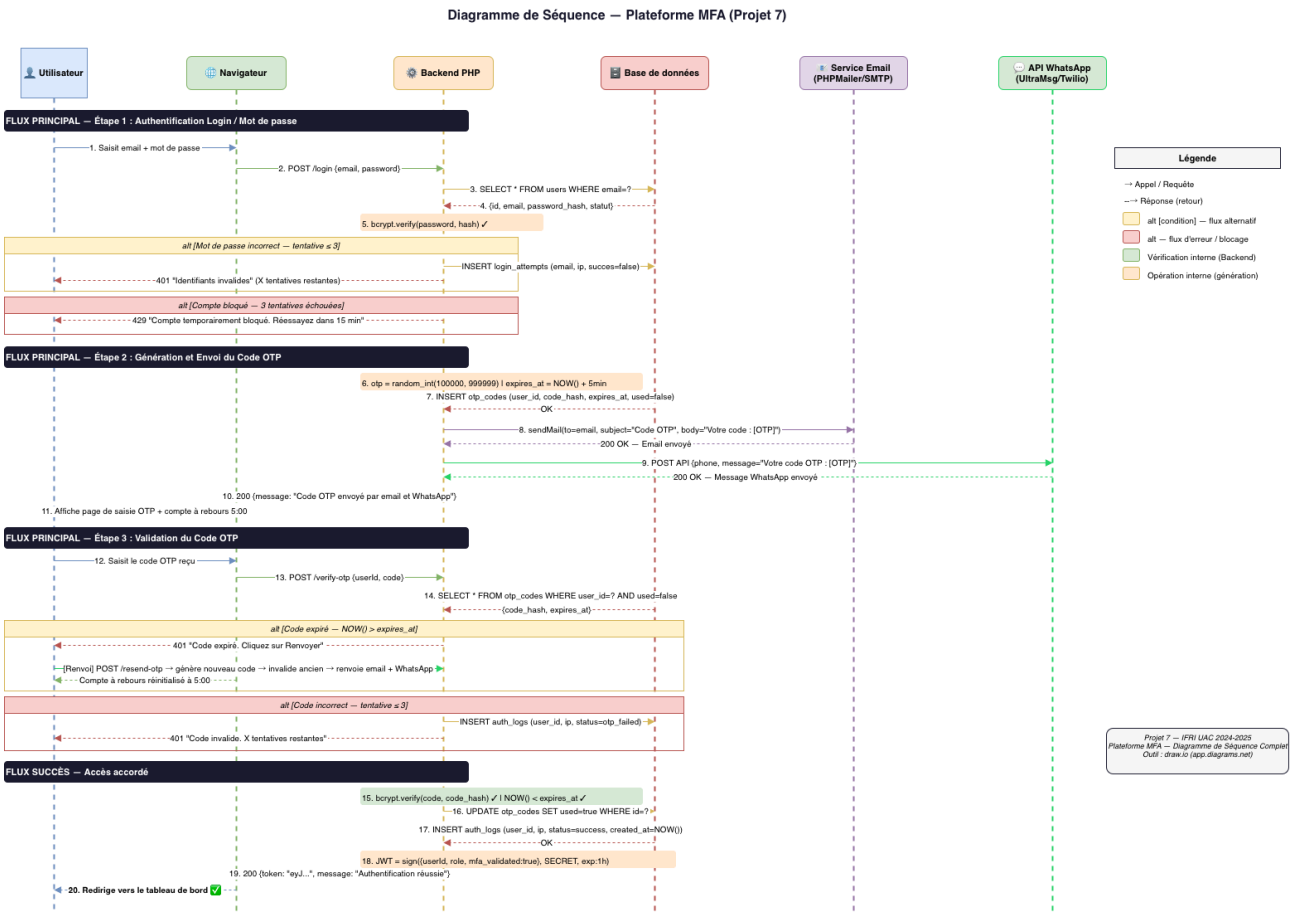


Figure 3 — Diagramme de séquence complet de la plateforme MFA

Acteurs représentés :

- Utilisateur — initie l'authentification
- Navigateur — interface web cliente
- Backend PHP — moteur de traitement et de vérification
- Base de données — stockage des credentials, OTP et logs
- Service Email (PHPMailer/SMTP) — envoi du code OTP par email
- API WhatsApp (UltraMsg/Twilio) — envoi du code OTP par WhatsApp

Flux couverts :

- Flux 1 — Succès complet : login → génération OTP → envoi dual → validation → JWT → dashboard
- Flux 2 — Erreur login : mot de passe incorrect avec compteur de tentatives et blocage à 3 échecs
- Flux 3 — Code OTP expiré : détection expiration + renvoi avec invalidation de l'ancien code
- Flux 4 — Code OTP incorrect : compteur de tentatives OTP avec blocage à 3 échecs

5. Product Backlog

ID	User Story	Critères d'acceptation	Priorité	Sprint	Points
US01	En tant qu'utilisateur, je veux m'inscrire sur la plateforme.	Formulaire avec email, login, mot de passe. Validation des champs. Email de confirmation.	**Haute**	S1	5
US02	En tant qu'utilisateur, je veux me connecter avec login et mot de passe.	Vérification des identifiants. Message d'erreur clair. Verrouillage après 3 échecs.	**Haute**	S2	8
US03	En tant qu'utilisateur, je veux recevoir un code OTP par e-mail.	Code reçu dans les 30 secondes. Code valide 5 minutes. Format 6 chiffres.	**Haute**	S3	8
US04	En tant qu'utilisateur, je veux recevoir un code OTP par WhatsApp.	Code reçu sur le numéro enregistré. Même délai de validité que l'e-mail.	**Haute**	S3	8
US05	En tant qu'utilisateur, je veux saisir le code OTP pour accéder à la plateforme.	3 tentatives max. Message d'erreur si code invalide. Redirection si succès.	**Haute**	S4	5
US06	En tant qu'utilisateur, je veux demander un renvoi du code OTP.	Bouton disponible après 60 secondes. Nouveau code invalide l'ancien.	**Moyenne**	S4	3
US07	En tant qu'administrateur, je veux consulter l'historique des connexions.	Tableau filtrable par date, utilisateur, statut, IP. Export possible.	**Haute**	S5	5
US08	En tant qu'administrateur, je veux gérer les comptes utilisateurs.	Créer, modifier, désactiver un compte. Réinitialisation de mot de passe.	**Haute**	S5	5
US09	En tant qu'utilisateur, je veux me déconnecter de façon sécurisée.	Session invalidée côté serveur. Redirection vers la page de connexion.	**Moyenne**	S6	2
US10	En tant qu'administrateur, je veux configurer les paramètres MFA.	Durée OTP, nombre de tentatives, canal par défaut configurables.	**Basse**	S6	3

Estimation totale : 52 points de complexité répartis sur 6 Sprints.

6. Planification détaillée des Sprints

Vue d'ensemble de la planification

Sprint	Intitulé	Fonctionnalité couverte	Durée	US
S1	**Mise en place & Inscription**	Environnement + Compte utilisateur	4 jours	US01
S2	**Authentification Phase 1**	Login + Mot de passe	4 jours	US02
S3	**Génération & Envoi OTP**	E-mail + WhatsApp	6 jours	US03, US04
S4	**Validation OTP & Accès**	Vérification code + Renvoi	5 jours	US05, US06
S5	**Module Administration**	Historique + Gestion comptes	6 jours	US07, US08
S6	**Finalisation & Déploiement**	Déconnexion + Config + Deploy	5 jours	US09, US10

Durée totale du projet : 30 jours.

Sprint 1 — Mise en place & Module d'inscription

SPRINT 1 — Mise en place & Module d'inscription	
Durée	4 jours
Objectif	Configurer l'environnement de développement, créer la base de données et développer le module d'inscription (création de compte utilisateur).
Sous-tâches	1. Installation de l'environnement (XAMPP/WAMP, PHP 8.x, MySQL) 2. Création de la base de données et des tables (applications, otpCodes) 3. Développement du formulaire d'inscription (HTML/CSS) 4. Traitement PHP côté serveur avec validation des champs 5. Hashage du mot de passe avec SHA256 6. Envoi de l'e-mail de confirmation d'inscription 7. Validation du compte via lien de confirmation 8. Page d'accueil et structure du projet (architecture MVC légère)
Langage / Justification	PHP 8.x (recommandé par le commanditaire, largement utilisé pour le développement web, nombreuses bibliothèques disponibles, PDO pour la sécurité des requêtes). MySQL pour la persistance des données. HTML5/CSS3 + Bootstrap 5 pour l'interface responsive. PHPMailer pour l'envoi d'e-mails.
Tests	Test unitaire : validation des fonctions d'inscription (vérification des champs obligatoires, format e-mail, longueur mot de passe). Test d'intégration : vérification que l'utilisateur est correctement enregistré en base de données avec mot de passe hashé et statut 'en attente de confirmation'.
Interfaces / Fenêtres	Page d'inscription (formulaire complet). Page de confirmation (message de succès). E-mail de confirmation reçu par l'utilisateur. Page d'accueil de la plateforme.

Use Case Sprint 1 : Utilisateur → S'inscrire sur la plateforme → [Système e-mail envoie confirmation]

Sprint 2 — Authentification Phase 1 (Login + Mot de passe)

SPRINT 2 — Authentification Phase 1 — Login & Mot de passe	
Durée	4 jours
Objectif	Développer le module de connexion par identifiant et mot de passe avec protection contre les attaques par force brute.
Sous-tâches	1. Développement du formulaire de connexion (login + mot de passe) 2. Vérification des identifiants en base de données (comparaison hash SHA256) 3. Gestion des sessions PHP sécurisées (session_regenerate_id après connexion) 4. Verrouillage du compte après 3 tentatives échouées consécutives 5. Journalisation des tentatives de connexion dans la table historique_connexions 6. Page de réinitialisation du mot de passe (mot de passe oublié) 7. Protection CSRF sur les formulaires 8. Redirection vers l'étape OTP en cas de succès
Langage / Justification	PHP 8.x avec sessions sécurisées (session_regenerate_id() après connexion). Utilisation de password_verify() natif PHP. PDO avec requêtes préparées pour prévenir les injections SQL. Protection CSRF avec tokens de formulaire.
Tests	Test unitaire : vérification de la fonction d'authentification (bon identifiant, mauvais identifiant, compte verrouillé). Test d'intégration : vérification que la session est correctement créée après connexion réussie et que l'utilisateur est redirigé vers la page OTP.
Interfaces / Fenêtres	Page de connexion (formulaire login/mot de passe). Message d'erreur en cas d'échec. Message de compte verrouillé. Page 'mot de passe oublié'. E-mail de réinitialisation du mot de passe.

Use Case Sprint 2 : Utilisateur → Se connecter (login + MDP) → [Succès] → Redirection vers validation OTP / [Échec] → Message d'erreur

Sprint 3 — Génération & Envoi du Code OTP

SPRINT 3 — Génération & Envoi du Code OTP (E-mail + WhatsApp)	
Durée	6 jours
Objectif	Développer le moteur de génération de codes OTP et les modules d'envoi par e-mail et par WhatsApp.
Sous-tâches	1. Développement de la fonction de génération OTP (6 chiffres, cryptographiquement sécurisé avec random_int()) 2. Enregistrement du code OTP en base avec sa date d'expiration (10 minutes) 3. Module d'envoi par e-mail via PHPMailer (configuration SMTP, template d'e-mail HTML) 4. Intégration de l'API WhatsApp (Whapi (gate.whapi.cloud)) 5. Gestion de l'envoi dual (e-mail ET WhatsApp simultanément) 6. Gestion des erreurs d'envoi (log des échecs, retry) 7. Page d'attente avec compte à rebours pour la durée de validité du code 8. Invalidation des anciens codes à chaque nouvelle génération
Langage / Justification	PHP 8.x avec random_int() pour la génération sécurisée. PHPMailer (bibliothèque PHP open-source pour l'envoi SMTP). API REST WhatsApp via UltraMsg ou Twilio (requêtes cURL en PHP). JSON pour le formatage des données envoyées à l'API.
Tests	Test unitaire : vérification que le code généré est bien à 6 chiffres et qu'il est unique en base. Test d'intégration : vérification que l'e-mail est bien reçu (avec une boîte de test type Mailtrap) et que l'API WhatsApp retourne un

	statut de succès. Test de non-régression : s'assurer que la génération d'un nouveau code invalide bien les précédents.
Interfaces / Fenêtres	Page de redirection post-connexion avec message 'Code envoyé à votre e-mail et WhatsApp'. Page de saisie du code OTP avec compte à rebours (5:00 → 0:00). Bouton 'Renvoyer le code' (désactivé pendant 60 secondes). Template e-mail HTML professionnel.

Use Case Sprint 3 : Système → Générer OTP → Envoyer par [Système e-mail] ET [API WhatsApp] → Utilisateur reçoit le code

Sprint 4 — Validation OTP & Accès à la plateforme

SPRINT 4 — Validation du Code OTP & Contrôle d'accès	
Durée	5 jours
Objectif	Développer le module de vérification du code OTP saisi par l'utilisateur et gérer l'accès à la plateforme.
Sous-tâches	1. Développement du formulaire de saisie du code OTP 2. Vérification du code (correspondance + expiration) 3. Gestion des tentatives erronées (max 3 tentatives puis blocage) 4. Fonctionnalité de renvoi du code (nouveau code, ancien invalidé) 5. Enregistrement du résultat de la validation OTP dans l'historique 6. Redirection vers le tableau de bord en cas de succès 7. Page d'accès refusé en cas d'échec définitif 8. Middleware de protection des pages (vérification de session + statut OTP validé) 9. Tests de la logique d'expiration (codes générés à t-6min doivent être rejetés)
Langage / Justification	PHP 8.x avec gestion précise des timestamps (comparaison NOW() > expire_at en MySQL). Utilisation des sessions pour maintenir l'état de l'authentification multi-étapes. JavaScript pour le compte à rebours côté client et la désactivation temporaire du bouton 'Renvoyer'.
Tests	Test unitaire : vérification avec un code correct non expiré (succès), un code incorrect (échec), un code expiré (rejet), et après 3 tentatives (blocage). Test d'intégration : vérification que l'utilisateur accède bien au tableau de bord après validation réussie et que les pages protégées sont inaccessibles sans OTP validé.
Interfaces / Fenêtres	Page de saisie du code OTP (6 champs individuels ou 1 champ à 6 caractères). Compte à rebours visible. Messages d'erreur (code invalide, code expiré, trop de tentatives). Page tableau de bord (accessible uniquement après MFA complet). Page d'accès refusé.

Use Case Sprint 4 : Utilisateur → Saisir code OTP → [Code valide] → Accéder à la plateforme / [Code invalide/expiré] → Message d'erreur / [Renvoi] → Générer nouveau code

Sprint 5 — Module d'Administration

SPRINT 5 — Module d'Administration	
Durée	6 jours
Objectif	Développer l'interface d'administration permettant la gestion des comptes et la consultation des historiques de connexion.
Sous-tâches	1. Interface de connexion administrateur (accès restreint par rôle) 2. Tableau de bord administrateur (statistiques globales : nb connexions, nb utilisateurs, taux d'échec) 3. Module de consultation de l'historique des connexions

	(tableau filtrable par date, utilisateur, IP, statut) 4. Export de l'historique en PDF et CSV 5. Module de gestion des comptes utilisateurs (liste, création, modification, désactivation) 6. Réinitialisation forcée du mot de passe d'un utilisateur 7. Vue détaillée d'un utilisateur (informations + historique personnel) 8. Pagination et recherche dans les listes
Langage / Justification	PHP 8.x avec vérification stricte des rôles (middleware admin). MySQL pour les requêtes filtrées et paginées. DataTables (bibliothèque JS) pour les tableaux interactifs côté client. TCPDF ou mPDF pour la génération des exports PDF. JavaScript/jQuery pour l'interactivité du tableau de bord.
Tests	Test unitaire : vérification que seul un utilisateur avec rôle 'admin' peut accéder aux pages d'administration (tentative avec compte normal doit retourner 403). Test d'intégration : vérification que le tableau de l'historique affiche correctement les données enregistrées lors des sprints précédents et que l'export PDF génère un fichier valide.
Interfaces / Fenêtres	Tableau de bord administrateur (statistiques, graphiques). Page liste des utilisateurs (avec filtres et pagination). Page détail utilisateur. Page historique des connexions (filtres, export). Formulaire de modification d'un compte. Interface de réinitialisation de mot de passe.

Use Case Sprint 5 : Administrateur → Consulter historique connexions / Gérer comptes utilisateurs / Voir statistiques → Exporter rapport

Sprint 6 — Finalisation, Configuration & Déploiement

SPRINT 6 — Finalisation, Configuration & Déploiement	
Durée	5 jours
Objectif	Finaliser la plateforme (déconnexion sécurisée, paramétrage MFA), effectuer les tests globaux, déployer en production et rédiger la documentation utilisateur.
Sous-tâches	1. Développement du module de déconnexion sécurisée (invalidation de session côté serveur) 2. Interface de configuration des paramètres MFA (durée OTP, nb max tentatives, canal par défaut) 3. Débogage global (révision de l'ensemble des sprints précédents) 4. Tests end-to-end (parcours utilisateur complet de l'inscription à l'accès) 5. Optimisation des performances (index BDD, cache si nécessaire) 6. Déploiement sur serveur de production (hébergement en ligne) 7. Configuration HTTPS (certificat SSL) 8. Rédaction du manuel utilisateur 9. Rédaction de la documentation technique
Langage / Justification	PHP 8.x avec session_destroy() et suppression des cookies de session. Interface de configuration stockée en base de données (table parametres). Déploiement via FTP ou hébergeur web (ex : InfinityFree, Hostinger, ou serveur VPS). Certificat SSL Let's Encrypt (gratuit).
Tests	Test unitaire : vérification que la déconnexion invalide bien la session (tentative d'accès après logout doit rediriger vers login). Test d'intégration complet (parcours bout-en-bout) : inscription → connexion → réception OTP → validation → accès tableau de bord → déconnexion. Test de sécurité : tentative d'accès direct aux URLs protégées sans authentification.
Interfaces / Fenêtres	Page de déconnexion (message de confirmation). Interface de configuration (formulaire admin). Page du manuel utilisateur (en ligne). Rapport de déploiement. Documentation technique du projet.

Use Case Sprint 6 : Utilisateur → Se déconnecter (session invalidée) / Administrateur → Configurer les paramètres MFA

7. Fin du projet

7.1 Débogage global

À l'issue des 6 Sprints, une phase de débogage global est conduite sur l'ensemble de la plateforme. Elle comprend :

- Révision systématique de chaque fonctionnalité développée.
- Correction des anomalies identifiées lors des tests end-to-end.
- Vérification de la cohérence des données en base de données.
- Contrôle des logs d'erreur PHP et des journaux du serveur.
- Tests de charge légers pour s'assurer de la stabilité.

7.2 Déploiement

Le déploiement de la plateforme se déroule en plusieurs étapes :

1. Préparation de l'environnement de production (hébergement, base de données).
2. Configuration du certificat SSL (HTTPS obligatoire).
3. Migration de la base de données (exécution des scripts SQL).
4. Upload des fichiers sur le serveur via FTP ou Git.
5. Tests de validation en environnement de production.
6. Mise en ligne officielle de la plateforme.

7.3 Maintenance

Un plan de maintenance minimal est prévu :

- Maintenance corrective : correction des bugs signalés après la mise en ligne.
- Maintenance évolutive : ajout de nouvelles fonctionnalités sur demande du commanditaire.
- Maintenance préventive : mise à jour des dépendances (PHPMailer, API WhatsApp), surveillance des logs de sécurité.
- Sauvegarde régulière de la base de données.

7.4 Manuel utilisateur

Un manuel utilisateur sera rédigé et mis à disposition en ligne. Il comprendra :

- Guide d'inscription et de première connexion.
- Procédure de validation du code OTP (e-mail et WhatsApp).
- Procédure de réinitialisation du mot de passe.
- Guide d'utilisation de l'interface administrateur.
- FAQ et résolution des problèmes courants.

8. Stack technique et outils

Catégorie	Technologie / Outil	Justification
Langage backend	**PHP 8.x**	Recommandé par le commanditaire, robuste, large écosystème, natif sur la plupart des hébergeurs.
Base de données	**MySQL / MariaDB**	SGBD relationnel fiable, compatible PHP, requêtes performantes pour l'historique.
Frontend	**HTML5 / CSS3 / Bootstrap 5**	Design responsive, composants UI prêts à l'emploi, compatibilité cross-navigateurs.
E-mail	**PHPMailer**	Bibliothèque PHP mature pour l'envoi SMTP, supporte TLS/SSL, facile à intégrer.
WhatsApp API	**Whapi (gate.whapi.cloud)**	API REST permettant l'envoi de messages WhatsApp via un numéro Business.
Serveur local	**XAMPP / WAMP**	Environnement de développement local intégrant Apache, MySQL et PHP.
Versioning	**Git + GitHub**	Gestion du code source, collaboration en équipe, traçabilité des modifications.
Gestion de projet	**Trello ou GitHub Projects**	Tableau Kanban SCRUM : Backlog, Sprint, In Progress, Done, Review.
Tests	**PHPUnit**	Framework de tests unitaires pour PHP, intégration facile avec le code existant.
PDF Export	**TCPDF ou mPDF**	Génération de rapports PDF côté serveur en PHP.

8.1 Architecture du système

Le Projet 7 est conçu comme une plateforme MFA-as-a-Service. Contrairement à un système d'auth classique, il fonctionne comme un service centralisé que les autres projets (1 à 6) consomment via une API REST. Chaque groupe s'inscrit, obtient un token API, puis appelle les endpoints pour envoyer et vérifier des OTP pour leurs utilisateurs.

Modèle de service

- Les autres groupes s'inscrivent sur la plateforme et reçoivent un token API unique
- Ce token (hashé en SHA256) est stocké dans la table applications
- Pour chaque utilisateur final à authentifier, le groupe appelle POST /api/send.php avec son token
- La plateforme génère un OTP, l'envoie par email (PHPMailer) ET WhatsApp (Whapi)
- Le groupe appelle ensuite POST /api/verify.php pour valider le code saisi

API WhatsApp — Whapi

La plateforme utilise l'API Whapi (gate.whapi.cloud) pour l'envoi des messages WhatsApp. L'authentification se fait via un Bearer token dans le header Authorization. La requête envoie le numéro de téléphone et le message contenant le code OTP.

Sécurité du token API

À l'inscription, un token API est généré et présenté une seule fois à l'administrateur. Ce token est hashé en SHA256 avant stockage en base de données. Lors de chaque appel API, le token envoyé est hashé et comparé au hash stocké — le token en clair ne transite donc jamais en base.

8.2 Documentation de l'API REST

La plateforme MFA expose deux endpoints REST consommables par les autres groupes. Tous les appels sont en JSON (Content-Type: application/json). Chaque appel doit inclure les credentials de l'application (email + token API) pour être authentifié.

Endpoints disponibles

Méthode	Endpoint	Description	Auth requise	Réponse succès
POST	/api/send.php	Générer et envoyer un OTP (email + WhatsApp)	email + token API	200 {success:true, email:true, whatsapp:true}
POST	/api/verify.php	Vérifier un code OTP soumis par un utilisateur	email + token API	200 {success:true, message:"Auth valide"}

Détail : POST /api/send.php — Envoyer un OTP

Cet endpoint génère un code OTP à 6 chiffres, l'enregistre en base avec une expiration de 10 minutes, puis l'envoie simultanément par email (PHPMailer/SMTP Gmail) et par WhatsApp (API Whapi). La réponse indique le statut d'envoi de chaque canal.

Corps de la requête (JSON) :

Champ JSON	Type	Description
email	string	Email de l'application inscrite sur la plateforme
token	string	Token API de l'application (en clair — sera hashé côté serveur pour comparaison)
userEmail	string	Email de l'utilisateur final qui recevra le code OTP
whatsapp	string	Numéro WhatsApp de l'utilisateur final (avec indicatif, ex: 22958000000)

Exemple de réponse succès :

{"success": true, "message": "Code OTP envoyé", "email": true, "whatsapp": true}

Détail : POST /api/verify.php — Vérifier un OTP

Cet endpoint vérifie qu'un code OTP soumis est correct, non expiré (expires_at > NOW()) et non déjà utilisé (is_used = 0). Si valide, il marque le code comme consommé (UPDATE is_used = 1) pour garantir l'usage unique.

Corps de la requête (JSON) :

Champ JSON	Type	Description
email	string	Email de l'application inscrite sur la plateforme
token	string	Token API de l'application
userEmail	string	Email de l'utilisateur dont on vérifie le code
otp	integer	Code OTP à 6 chiffres saisi par l'utilisateur

Exemple de réponse succès :

{"success": true, "message": "Auth valide"}

Codes de réponse HTTP

Code HTTP	Cas	Message retourné
200	Succès	{success:true, message:"Auth valide"} ou {success:true, message:"Code OTP envoyé"}
400	Champs manquants	{success:false, message:"Veuillez renseigner correctement les clés"}

401	Token invalide	{success:false, message:"Token incorrect"}
401	Code incorrect	{success:false, message:"Code incorrect"}
401	Email inconnu	{success:false, message:"Votre email n'est pas éligible"}
422	Code expiré	{success:false, message:"Code expiré"}
422	Code déjà utilisé	{success:false, message:"Code déjà utilisé"}
500	Erreur envoi	{success:false, message:"Échec de l'envoi", errors:[...]}

Exemple d'intégration par un autre groupe

Voici comment le Projet 1 (Module d'inscription) peut utiliser cette API pour ajouter une couche MFA à sa connexion :

- Étape 1 — Après validation du mot de passe, appeler POST /api/send.php avec l'email et le WhatsApp de l'utilisateur
- Étape 2 — Afficher un formulaire de saisie du code OTP à l'utilisateur
- Étape 3 — Appeler POST /api/verify.php avec le code saisi
- Étape 4 — Si {success: true}, accorder l'accès ; sinon afficher le message d'erreur retourné

9. Schéma de la base de données

La base de données est composée de deux tables principales correspondant exactement au code implémenté. La table applications stocke les comptes des développeurs/groupes qui utilisent l'API. La table otpCodes stocke chaque code OTP généré avec son contexte d'utilisation.

Table	Colonne	Type SQL	Contrainte	Description
applications	id	INT	PK AI	Identifiant unique de l'application inscrite
	username	VARCHAR(100)	NOT NULL	Nom de l'application / développeur
	email	VARCHAR(150)	UNIQUE NOT NULL	Email de connexion au tableau de bord
	password	VARCHAR(255)	NOT NULL	Mot de passe hashé en SHA256
	token	VARCHAR(255)	UNIQUE NOT NULL	Token API hashé en SHA256 — utilisé pour authentifier les appels API
	is_active	TINYINT(1)	DEFAULT 1	1 = compte actif, 0 = suspendu
	created_at	TIMESTAMP	DEFAULT NOW()	Date de création du compte
otpCodes	id	INT	PK AI	Identifiant unique du code OTP

	whatsapp	VARCHAR(20)	NOT NULL	Numéro WhatsApp destinataire (avec indicatif)
	user_email	VARCHAR(150)	NOT NULL	Email de l'utilisateur final à authentifier
	otp	VARCHAR(10)	NOT NULL	Code OTP à 6 chiffres (stocké en clair)
	expires_at	TIMESTAMP	NOT NULL	Expiration : NOW() + INTERVAL 10 MINUTE
	app_id	INT	FK → applications.id	Référence vers l'application qui a généré le code
	is_used	TINYINT(1)	DEFAULT 0	0 = non utilisé, 1 = code déjà consommé (usage unique)
	created_at	TIMESTAMP	DEFAULT NOW()	Horodatage de génération du code

Relation entre les tables

- applications (1) ──< (N) otpCodes : une application peut générer plusieurs codes OTP pour ses utilisateurs
- La colonne app_id dans otpCodes référence l'id de l'application ayant demandé la génération
- La colonne is_used garantit l'usage unique de chaque code (1 = consommé, ne peut plus être réutilisé)
- La colonne expires_at permet la vérification d'expiration via la requête : expires_at < NOW()

Note sur la sécurité du stockage OTP

Dans la version actuelle (V1), le code OTP est stocké en clair dans la base de données. Cette approche est fonctionnelle pour la démonstration. La Version 2 du projet prévoira le hashage des codes OTP avant stockage pour renforcer la sécurité en cas de compromission de la base.

10. Gestion des risques

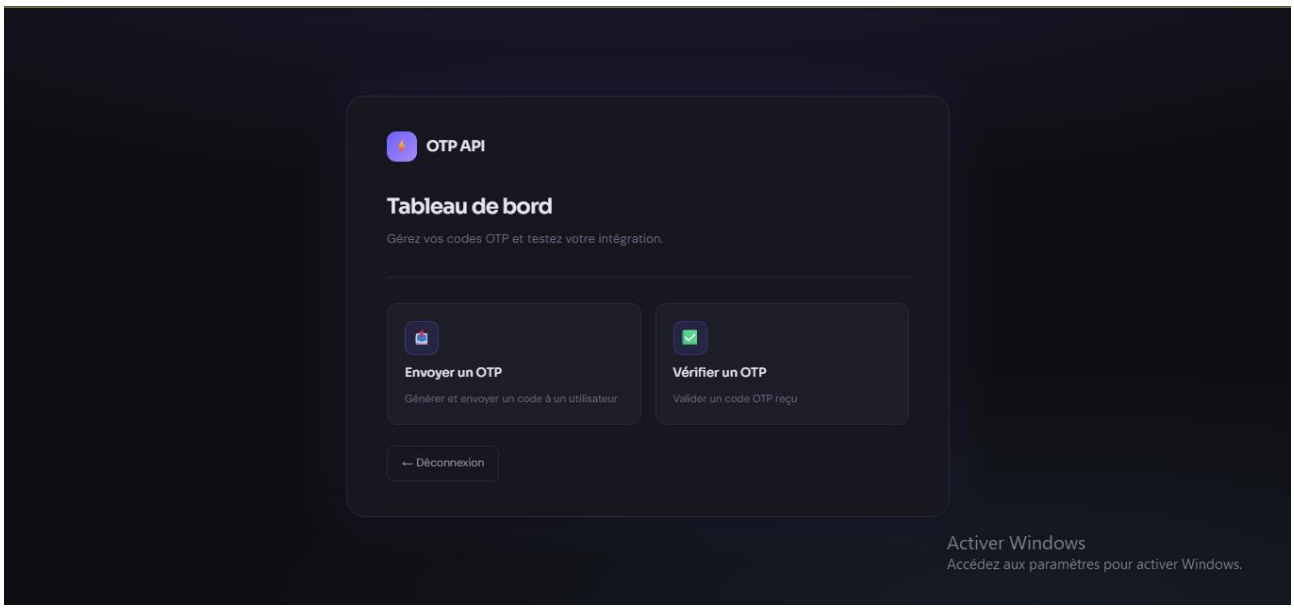
L e tableau ci-dessous recense les risques identifiés pour le Projet 7, mis à jour pour refléter l'état réel du développement. Les risques déjà atténués par les choix techniques effectués (Whapi intégré, token hashé, is_used) sont signalés.

Risque identifié	Probabilité	Impact	Mesure d'atténuation
Dépassement du délai de 30 jours	Moyenne	Élevé	Priorisation stricte du backlog SCRUM ; réduction du périmètre si nécessaire ; suivi quotidien en Daily Scrum
Difficultés d'intégration de l'API Whapi	Faible	Moyen	L'API Whapi est déjà intégrée et fonctionnelle. Solution de repli : envoi email uniquement si WhatsApp échoue (déjà géré dans send.php)
Token API compromis	Faible	Élevé	Le token est hashé en SHA256 avant stockage. En cas de compromission, l'admin peut régénérer un nouveau token depuis le tableau de bord
OTP intercepté (stockage en clair)	Faible	Moyen	Le code expire après 10 minutes et ne peut être utilisé qu'une seule fois (is_used). La V2 prévoira le hashage des OTP en base
Quota SMTP Gmail dépassé	Moyenne	Moyen	Utiliser Mailtrap en développement. Prévoir un compte Gmail dédié avec App Password. Le canal WhatsApp reste fonctionnel même si l'email échoue
Conflits Git entre développeurs	Élevée	Moyen	Convention de nommage des branches (feature/sprint-X) ; pull requests obligatoires avant merge sur main ; merges quotidiens
Indisponibilité d'un membre	Faible	Moyen	Documentation des tâches dans le backlog Trello ; binômage sur les tâches critiques ; réaffectation par le Scrum Master

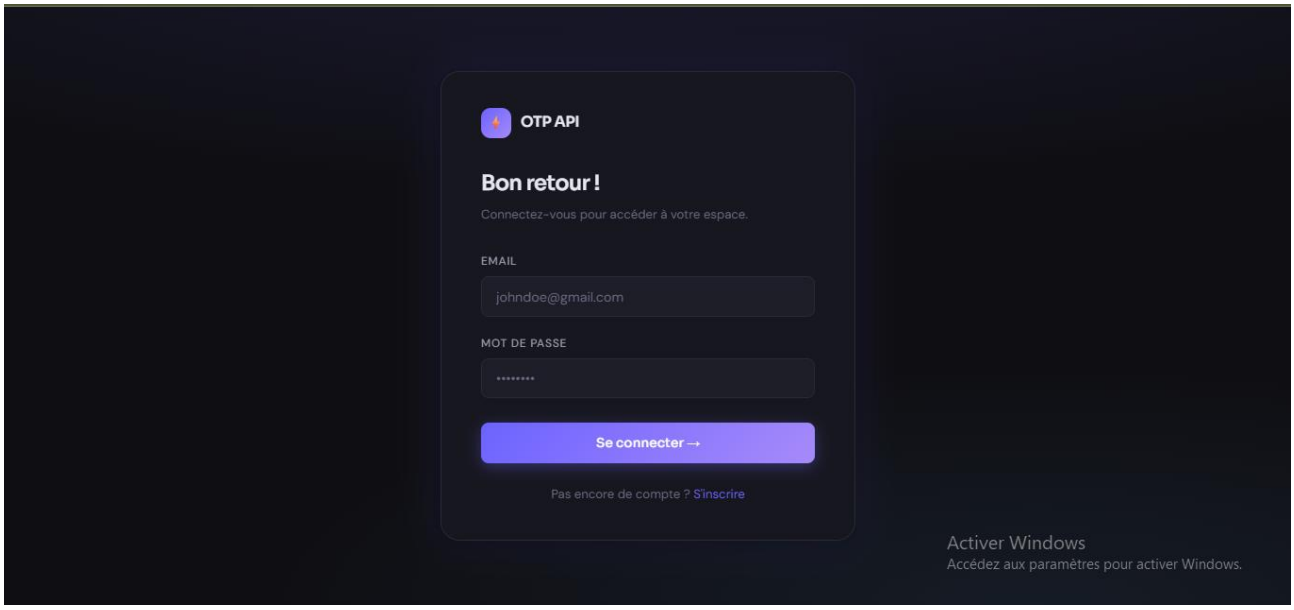
Ces risques sont réévalués à chaque Sprint Retrospective. Tout nouveau risque est ajouté lors du Sprint Planning suivant.

11. Captures

1 – Dashboard



2 – Login



3 – Send_test_postman

POST http://localhost:81/DoubleAuth/api/send.php Send

Docs Params Authorization Headers (9) Body Scripts Settings Cookies

none form-data x-www-form-urlencoded raw binary GraphQL JSON Schema Beautify

```
1 {
2     "email": "fadilmoussi1250@gmail.com",
3     "token": "28da2b3e50ea2d5cfe070a9078ab00846061c8f52a02bf6493a0391f2bfcb147",
4     "whatsapp": "22967548463",
5     "userEmail" : "fadilmoussi1250@gmail.com"
6 }
```

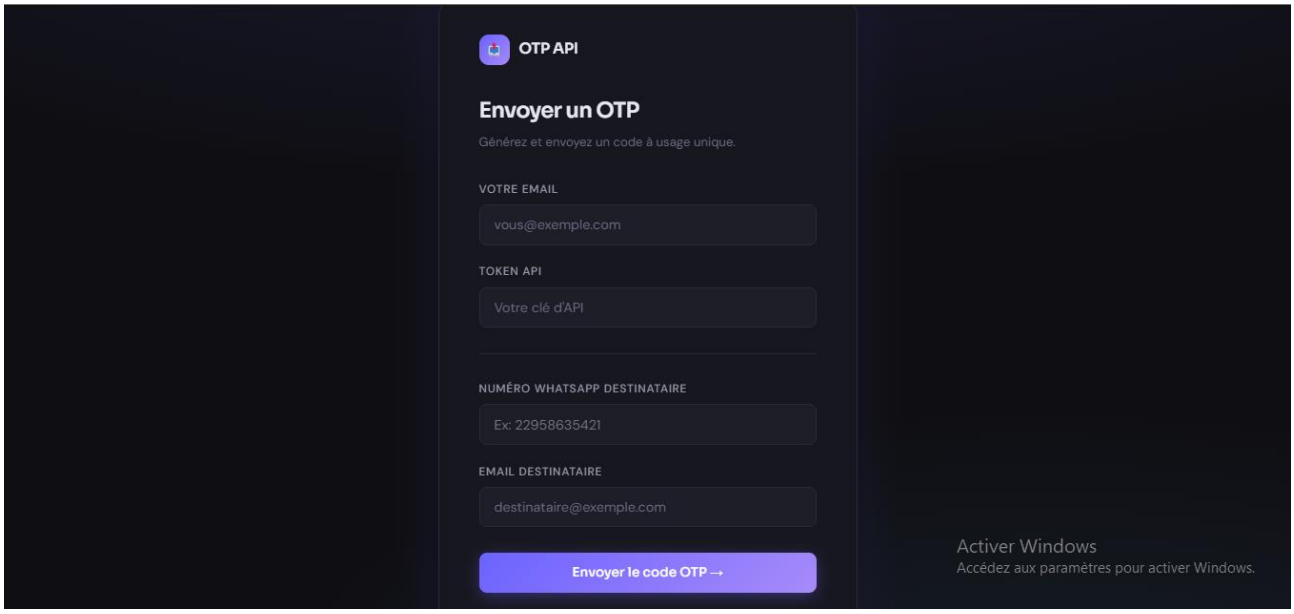
Body Cookies Headers (9) Test Results 200 OK 6.09 s 394 B

{} JSON Preview Visualize

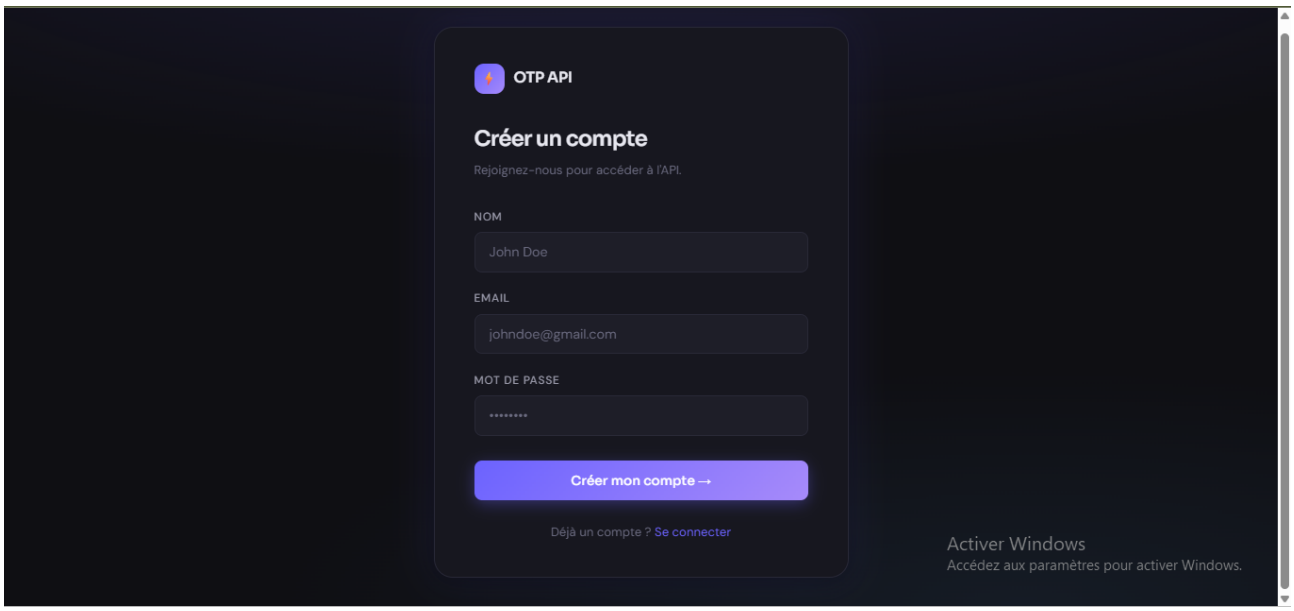
```
1 {
2     "success": true,
3     "message": "Code OTP envoyé",
4     "email": true,
5     "whatsapp": true
6 }
```

Activer Windows
Accédez aux paramètres pour activer Window

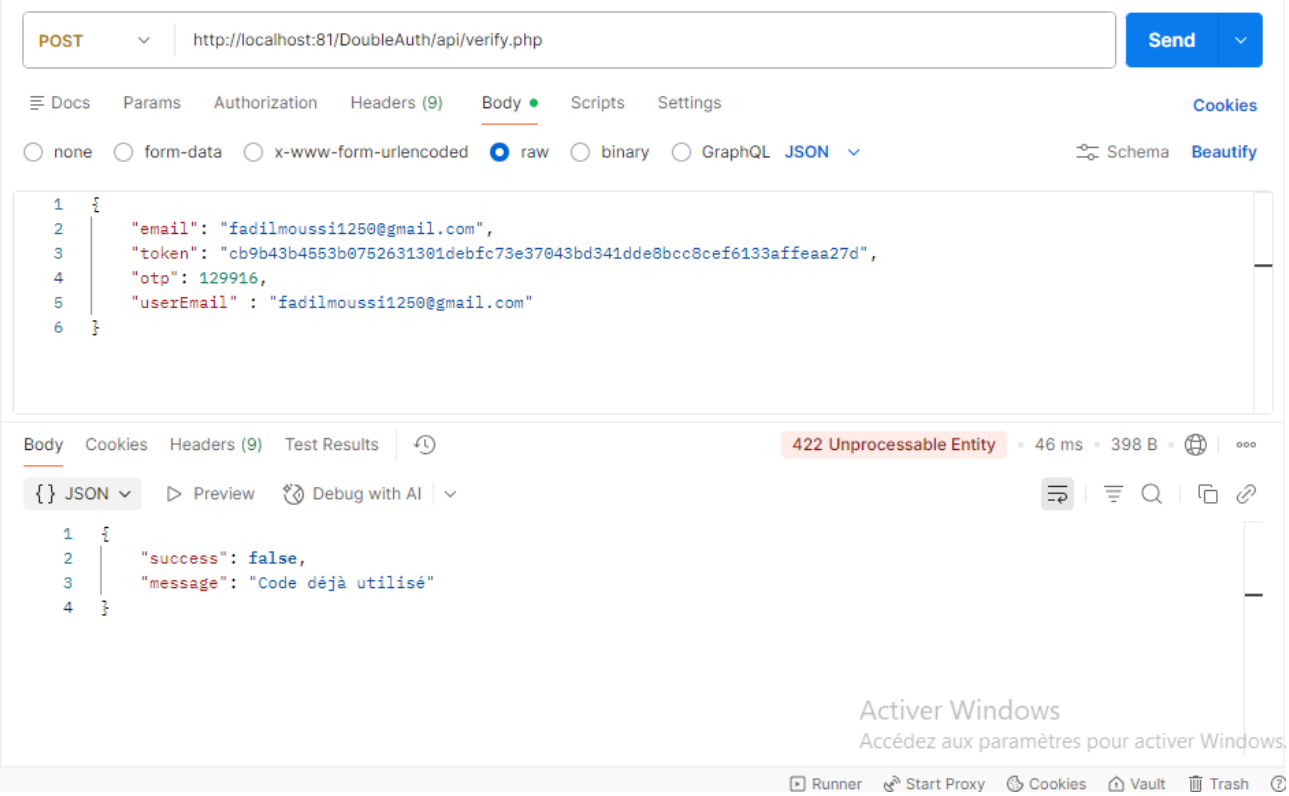
4 – send_view



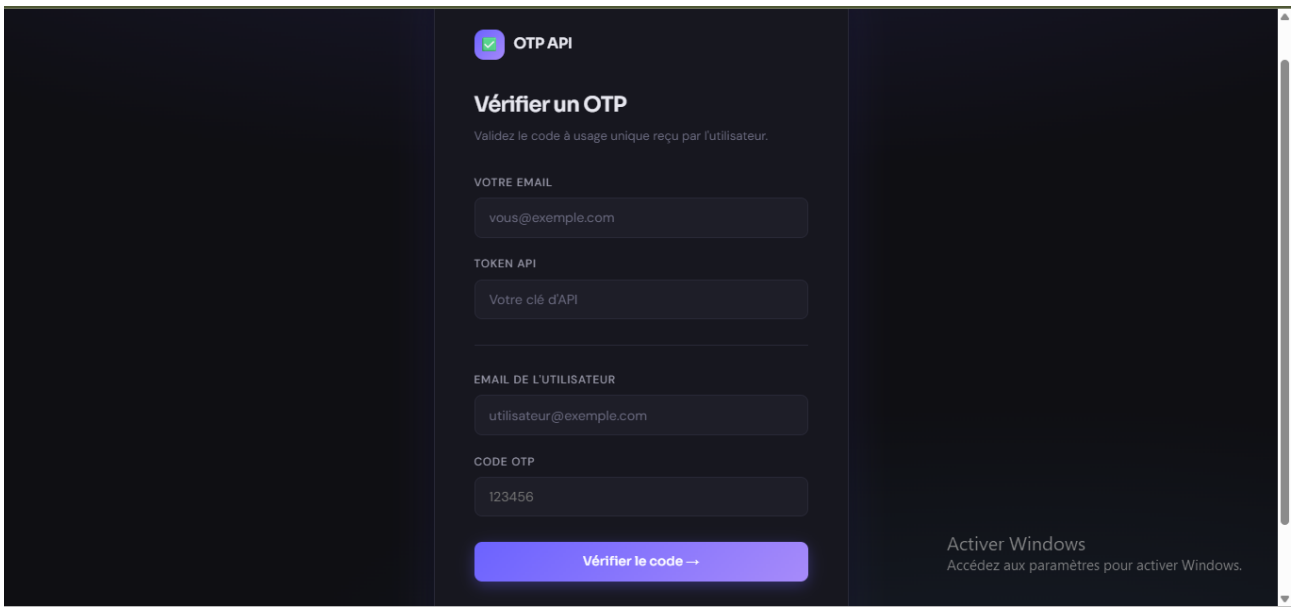
5 – signup



6 – verify_test_postman



7 – verify_view



12. Annexe

Tout le code source ainsi que les captures d'écran est disponible sur ce repository
https://github.com/shanks-em/2FA-platform

13. Conclusion

Ce cahier des charges présente une vision complète et structurée du Projet 7— Plateforme d'Authentification Multi-Facteurs. En suivant la méthodologie SCRUM, le projet sera développé de façon itérative et incrémentale, garantissant des livraisons régulières et une validation continue par le commanditaire.

Les 6 Sprints planifiés couvrent l'intégralité des fonctionnalités requises, de l'inscription à la déconnexion sécurisée, en passant par la génération et la validation des codes OTP via e-mail et WhatsApp, ainsi que le module d'administration complet.

La durée totale du projet est de 30 jours, avec 52 points de complexité répartis équitablement sur les sprints. La plateforme sera développée en PHP 8.x avec MySQL, et déployée avec HTTPS pour garantir la sécurité des échanges.

Fait à Abomey-Calavi, Mars 2025